

Laboratorio IA 1: Evolución en consola

Materia: Inteligencia Artificial

Dinámica: práctica en terminal (hands-on) **Entregable:** repositorio individual de GitHub

Comprobar mediante código cómo el comportamiento inteligente (estrategia y resolución de problemas) emerge a través de la evolución y la interacción con un entorno simulado, sin haber sido programado de forma manual. La práctica ejemplifica la teoría de morfología evolutiva de Karl Sims discutida en la clase teórica.

Antes de comenzar

Necesitarás:

- un repositorio de GitHub asignado para esta clase;
- Python 3 instalado;
- una terminal o consola de comandos;
- un editor de código, como VS Code o Sublime Text.

1. Preparar el entorno

1. Abre tu terminal o consola de comandos (`cmd` en Windows, Terminal en Linux/Mac).
2. Navega hasta la carpeta de tu repositorio de GitHub.
3. Crea una subcarpeta llamada `Lab_1_Evolucion` y entra en ella:

```
mkdir Lab_1_Evolucion
cd Lab_1_Evolucion
```

4. Verifica la instalación de Python:

```
python3 --version
# En Windows tambien puedes probar: python --version
```

5. Abre la carpeta en tu editor de código.

2. Calentamiento evolutivo: motor de texto

El archivo `texto.py` se entrega junto con este PDF. Descárgalo o cópialo directamente al repositorio; así no tendrás que copiar código desde el PDF. Si necesitas revisarlo, el código aparece también a continuación. Ejecútalo con `python3 texto.py` (o `python texto.py`).

Advertencia importante

La variable `objetivo` solo funciona con letras mayúsculas y espacios. Si incluyes minúsculas, números o caracteres especiales, el programa no encontrará la coincidencia y entrará en un bucle infinito.

```
import random
import time
import string
import os

def limpiar_pantalla():
    os.system('cls' if os.name == 'nt' else 'clear')

# El entorno (La palabra a evolucionar)
# REGLA DE ORO: Solo mayusculas y espacios
objetivo = "ROBOTICA COGNITIVA"
letras_posibles = string.ascii_uppercase + " "

# El azar inicial (ADN basura)
individuo = "".join(random.choice(letras_posibles) for _ in range(len(objetivo)))
generacion = 0

# El motor evolutivo
while individuo != objetivo:
    nuevo_individuo = ""
    for i in range(len(objetivo)):
        # Seleccion natural a nivel de caracter
        if individuo[i] == objetivo[i]:
            nuevo_individuo += individuo[i]
        else:
            nuevo_individuo += random.choice(letras_posibles)

    individuo = nuevo_individuo
    generacion += 1

    limpiar_pantalla()
    print(f"Gen {generacion}: {individuo}")
    time.sleep(0.05)

print(f"\nEvolucion completada en {generacion} generaciones!")
```

Prueba rápida: modifica la variable `objetivo` con tu nombre completo, usando solo mayúsculas y espacios, y observa cuántas generaciones requiere la simulación para converger.

3. Interacción espacial: el robot 2D

Ahora aplicaremos la evolución al espacio físico bidimensional. El archivo `robot.py` se entrega junto con este PDF para evitar problemas de copiado e indentación. Cópialo directamente al repositorio y ejecútalo. El agente aprende a navegar hacia el cubo basándose exclusivamente en un sistema de recompensas (*fitness function*).

```
import random
import time
import os

def limpiar_pantalla():
    os.system('cls' if os.name == 'nt' else 'clear')
```

```

# =====
# ZONA DE TRABAJO (LA REGLA DE SUPERVIVENCIA)
# =====
def evaluar_robot(adn):
    posicion = [0, 0] # Coordenadas iniciales: [Fila, Columna]

    # El robot ejecuta su secuencia genetica a ciegas
    for comando in adn:
        if comando == 'U': posicion[0] -= 1
        elif comando == 'D': posicion[0] += 1
        elif comando == 'L': posicion[1] -= 1
        elif comando == 'R': posicion[1] += 1

    cubo = [7, 7] # Coordenadas de la meta
    distancia = abs(cubo[0] - posicion[0]) + abs(cubo[1] - posicion[1])

    # El fitness base premia la proximidad a la meta
    puntaje = 100 - distancia
    return puntaje

# =====
# MOTOR GRAFICO (NO MODIFICAR)
# =====
def animar_mejor_robot(adn, generacion, puntaje):
    posicion, cubo, tamano = [0, 0], [7, 7], 8

    for paso, comando in enumerate(adn):
        limpiar_pantalla()
        print(f"Gen {generacion} | Puntos: {puntaje}/100")

        if comando == 'U': posicion[0] -= 1
        elif comando == 'D': posicion[0] += 1
        elif comando == 'L': posicion[1] -= 1
        elif comando == 'R': posicion[1] += 1

        posicion[0] = max(0, min(posicion[0], tamano - 1))
        posicion[1] = max(0, min(posicion[1], tamano - 1))

        for fila in range(tamano):
            linea = ""
            for col in range(tamano):
                if fila == cubo[0] and col == cubo[1]:
                    linea += "[META]" if posicion == cubo else "[CUBO]"
                elif fila == posicion[0] and col == posicion[1]:
                    linea += "[ROBOT]"
                else:
                    linea += "[ ]"
            print(linea)
            time.sleep(0.08)

# =====
# CICLO EVOLUTIVO (NO MODIFICAR)
# =====
comandos = ['U', 'D', 'L', 'R']
mejor_robot = "".join(random.choice(comandos) for _ in range(30))
generacion = 0

while evaluar_robot(mejor_robot) < 100:
    robot_mutante = ""

```

```

for gen in mejor_robot:
    robot_mutante += random.choice(comandos) if random.random() < 0.15 else gen

    if evaluar_robot(robot_mutante) > evaluar_robot(mejor_robot):
        mejor_robot = robot_mutante

    generacion += 1
    # Se anima el progreso cada 150 iteraciones
    if generacion % 150 == 0:
        animar_mejor_robot(mejor_robot, generacion, evaluar_robot(mejor_robot))

animar_mejor_robot(mejor_robot, generacion, evaluar_robot(mejor_robot))

```

Nota sobre el copiado: los espacios iniciales son importantes en Python. Este documento conserva la indentación en los bloques de código; después de pegar, comprueba visualmente que las líneas dentro de cada función, bucle y condición estén desplazadas hacia la derecha. Si tu lector de PDF elimina esos espacios, utiliza el archivo fuente del repositorio en lugar de copiar desde el PDF.

El archivo Python conserva los emojis del material original. Si tu terminal no los muestra, puedes sustituirlos temporalmente por etiquetas de texto como [ROBOT], [CUBO] y [META]; la lógica del ejercicio no cambia.

4. Reto de laboratorio

El código base es funcional, pero el entorno carece de presión selectiva compleja. Modifica **única-mente** la función `evaluar_robot(adn)` en `robot.py` para resolver uno de los siguientes escenarios.

Existe un pozo de ácido letal en la coordenada exacta [4, 4].

Implementación: modifica la iteración dentro de `evaluar_robot`. Si en cualquier punto de la trayectoria el robot pisa el ácido (`posicion == [4, 4]`), debe recibir una penalización masiva, por ejemplo, restar 50 puntos a su puntaje final.

Señal de éxito: la terminal mostrará cómo la presión evolutiva obliga al robot a alterar su ADN para rodear el centro del tablero.

En sistemas biológicos reales, el movimiento consume energía vital.

Implementación: el ADN consta de 30 instrucciones. En la versión actual, el agente desperdicia movimientos y da vueltas innecesarias antes de llegar a la meta. Modifica el cálculo del puntaje para penalizar la ineficiencia, restando puntos por comandos superfluos. El camino óptimo al objetivo toma exactamente 14 movimientos útiles.

Señal de éxito: la evolución descartará las secuencias ineficientes. El resultado final será un agente que se desplaza en línea recta sin dudar y luego permanece deliberadamente estático contra los límites del tablero para conservar energía.

5. Entrega

1. Añade comentarios breves en la función `evaluar_robot` explicando la lógica utilizada para resolver el Escenario A o B.
2. Abre la terminal en la carpeta del repositorio y agrega los archivos modificados:

```
git add .
```

3. Crea un commit descriptivo:

```
git commit -m "Agrega scripts de evolucion algoritmica para el Laboratorio 1"
```

4. Sube los cambios a tu repositorio:

```
git push
```

5. Verifica en GitHub que `texto.py` y `robot.py` estén en la carpeta correcta. El profesor y el asistente revisarán los avances directamente desde allí.

Lista de comprobación

- ☐ Creé la carpeta `Lab_1_Evolucion`.
- ☐ Ejecuté `texto.py` y probé un objetivo propio.
- ☐ Ejecuté `robot.py`.
- ☐ Modifiqué solo `evaluar_robot(adn)`.
- ☐ Comenté la lógica de mi solución.
- ☐ Hice `git add`, `git commit` y `git push`.
- ☐ Verifiqué los archivos en GitHub.